WebSockets Security

Complete Pentesting & Security Testing Guide

Reference: PortSwigger Web Security Academy

A consolidated study and practical security report covering WebSocket fundamentals, protocol behavior, attack surface, Burp Suite testing, message manipulation, handshake attacks, Cross-Site WebSocket Hijacking, and defensive controls.

1. Introduction to WebSockets

WebSockets provide persistent, bidirectional, full-duplex communication between a client, usually a browser, and a server over a single TCP connection. Unlike traditional HTTP, WebSockets allow either side to send data asynchronously at any time after the connection is established.

They are optimized for real-time applications where low latency and continuous data exchange are important.

• Live chat applications

• Real-time notifications and dashboards

• Online gaming and streaming

• Financial trading and market data feeds

• Collaborative tools

2. WebSocket Protocol Overview

WebSockets are standardized in RFC 6455. Communication starts as HTTP and uses an Upgrade handshake to switch the connection to the WebSocket protocol.

Key Characteristics

• Persistent connection

• Full-duplex and bidirectional communication

• Low latency

• Message-based communication rather than request-response

• Support for text, binary, and control frames

3. HTTP vs WebSockets

Characteristic	HTTP	WebSockets
Communication	Request-response	Full-duplex
State	Stateless	Stateful connection
Initiation	Client initiates	Client or server can send
Connection	Opened/reused per requests	Single persistent connection
Real-time overhead	Higher	Lower

This architectural difference creates a distinct attack surface. Security controls commonly designed around HTTP requests may not automatically protect individual WebSocket messages.

4. Establishing a WebSocket Connection

The client sends an HTTP request asking the server to upgrade the connection.

Handshake Request

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
Origin: https://example.com
```

Handshake Response

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

After the 101 Switching Protocols response, communication continues using WebSocket frames.

5. WebSocket Messages and Frames

WebSocket messages may contain text, JSON, binary data, or application-specific formats. Messages often resemble API requests.

```
{
"action": "sendMessage",
"message": "Hello"
}
```

The server must validate every message. A trusted connection does not mean every message sent over that connection is authorized or safe.

6. WebSocket Security Attack Surface

• Missing or weak authentication

• Missing per-message authorization

• Insecure message handling and injection

• Excessive trust in client-side controls

• Improper Origin validation

• Cross-Site WebSocket Hijacking (CSWSH)

• Replay and message flooding

• Unencrypted ws:// transport

Traditional HTTP-only testing is insufficient. A tester should inspect both the initial handshake and the messages exchanged after the protocol upgrade.

7. Testing WebSockets with Burp Suite

Burp Suite can be used to intercept the handshake, review WebSocket history, modify messages, replay captured traffic, and generate new messages.

Typical Testing Workflow

• Browse to the WebSocket-enabled feature.

• Capture a normal WebSocket message.

• Review message structure and action names.

• Modify parameters or payload values.

• Replay the message and observe server behavior.

• Craft actions that are not exposed by the user interface.

• Test the handshake by changing security-sensitive headers.

8. Manipulating WebSocket Messages

Intercepting and modifying message fields can reveal missing server-side validation or authorization.

Example: Parameter Manipulation

```
Original:
{"action":"updateEmail","email":"user@example.com"}

Modified:
{"action":"updateEmail","email":"attacker@example.com"}
```

If the server processes the modified action without enforcing the current user's permissions and intended workflow, the endpoint may be vulnerable.

9. WebSocket-Based XSS

If WebSocket message content is rendered in another user's browser without proper output encoding or sanitization, an attacker may be able to inject client-side content.

Example Test Payload

```
<img src=1 onerror='alert(1)'>
```

Testing Scenario

• Open a live chat feature.

• Intercept a WebSocket message.

• Replace normal message content with a controlled test payload.

• Forward the message.

• Observe whether unsafe HTML is rendered in the receiving context.

Potential impact: client-side code execution in the affected rendering context. Testing should only be performed in authorized labs or environments.

10. Replaying and Generating New Messages

Attackers are not limited to actions available in the visible user interface. Captured messages can be replayed and new messages can be crafted.

```
{"action":"deleteUser","userId":123}
```

A server that trusts client-side controls may process hidden or unauthorized actions. Test sensitive actions with a low-privileged account and verify that authorization is enforced for every message.

Privilege Escalation Test Example

```
{"action":"changeRole","role":"admin"}
```

11. Manipulating the WebSocket Handshake

The handshake is a critical security boundary. Test how the server behaves when security-relevant headers are changed or removed.

Common Tests

• Remove the Origin header.

• Replace Origin with an untrusted test origin.

• Review authentication cookies or tokens.

• Check whether expired or invalid sessions can connect.

• Test malformed or unexpected handshake headers in an authorized environment.

```
Origin: https://evil.example
```

Weak handshake validation can expose unauthorized WebSocket access or contribute to cross-site attacks.

12. Cross-Site WebSocket Hijacking (CSWSH)

CSWSH occurs when a victim's browser can be induced to establish a WebSocket connection to a trusted application using the victim's authenticated context. Risk is especially significant when authentication relies on cookies and the server does not strictly validate Origin.

Common Conditions

• The endpoint relies on cookies for authentication.

• The browser includes the authenticated session during the handshake.

• Origin validation is missing or weak.

• No additional unpredictable token or equivalent handshake protection is required.

Conceptual Test

```
var ws = new WebSocket("wss://target.example/chat");

ws.onopen = function () {
ws.send("READY");
};

ws.onmessage = function (event) {
console.log(event.data);
};
```

If an untrusted origin can establish an authenticated connection and read or trigger sensitive operations, the application may be vulnerable to CSWSH.

13. Authentication and Authorization Testing

Authentication should establish who is connected, while authorization should determine whether that identity may perform each requested action.

• Require valid authentication during connection establishment.

• Do not rely solely on client-side state.

• Validate authorization for every message and action.

• Verify object ownership when identifiers are supplied.

• Retest sensitive actions using low-privileged accounts.

• Handle session expiration and logout for active connections.

14. Input Validation and Message Security

Treat every WebSocket message as untrusted input. The same vulnerability classes that affect HTTP APIs can also appear in WebSocket handlers.

• Validate message schema and data types.

• Allowlist supported action names.

• Validate length and format constraints.

• Encode output according to the rendering context.

• Use parameterized database operations.

• Reject unexpected fields when appropriate.

• Handle malformed JSON and binary frames safely.

15. Rate Limiting, Replay, and Availability

Persistent connections can be abused for high-volume message delivery, brute force attempts, or repeated execution of sensitive operations.

• Apply per-user and per-connection rate limits.

• Limit message size.

• Set connection and idle timeouts.

• Detect repeated sensitive actions.

• Use replay protections where operations must be one-time.

• Monitor abnormal connection and message volume.

16. Secure Transport

Use secure WebSockets through wss:// so WebSocket traffic is protected by TLS. Plaintext ws:// connections can expose messages to interception or modification on untrusted networks.

• Use wss:// in production.

• Use valid TLS certificates.

• Avoid mixed-content configurations.

• Review proxy and load balancer TLS termination.

• Protect authentication tokens and session identifiers.

17. WebSocket Pentesting Cheat Sheet

Test Area	Checks
Messages	Modify JSON fields; change action values; test types and lengths
Replay	Resend captured messages; repeat sensitive actions
New messages	Craft hidden or undocumented actions
Authorization	Repeat privileged actions as a low-privileged user
Handshake	Remove/change Origin; review cookies and tokens
CSWSH	Attempt connection from an untrusted test origin
Input	Test controlled injection strings and malformed structures
Transport	Check ws:// vs wss:// and TLS handling
Availability	Review rate limits, message size, and connection limits

18. Example Payload and Impact Summary

Vulnerability Type	Example	Potential Impact
WebSocket XSS	<img src=1 onerror='alert(1)'>	Client-side code execution
Authorization bypass	{"action":"changeRole","role":"admin"}	Privilege escalation
Hidden action	{"action":"deleteUser","userId":123}	Unauthorized operation
Handshake weakness	Modified Origin header	Unauthorized cross-site connection
CSWSH	Cross-origin WebSocket connection	Authenticated data/action abuse

19. Recommended Defenses

• Strictly validate the Origin header during the handshake.

• Use wss:// for encrypted transport.

• Require strong authentication.

• Avoid relying solely on cookies when additional handshake protection is appropriate.

• Enforce authorization for every message.

• Validate and sanitize message content.

• Use allowlists for actions and parameters.

• Implement rate limiting and message-size limits.

• Log connection, authentication, and sensitive action events.

• Close or invalidate connections when sessions expire or users log out.

20. Conclusion

WebSockets enable efficient real-time communication but expand the attack surface of modern applications. Security must cover both the HTTP upgrade handshake and every message exchanged after the connection is established.

For penetration testers, the core methodology is to intercept, understand, modify, replay, and generate messages; test authorization with different privilege levels; manipulate the handshake; and assess whether an untrusted origin can establish an authenticated connection.

Key takeaway: treat WebSocket endpoints with the same security rigor as APIs. Authenticate connections, authorize every sensitive action, validate all input, enforce trusted origins, use encrypted transport, and monitor abuse.

21. Reference

Primary learning reference: PortSwigger Web Security Academy — WebSockets.

This consolidated report combines the strongest conceptual, practical lab, cheat-sheet, and defensive material from the three provided WebSocket study documents into one structured final version.